

8. Create a basic 2D character controller. Move the player character using Unity's Input System.

✓ Step 1: Add Player

1. Go to **Hierarchy (left panel)**
2. **Right-click** → **2D Object** → **Sprites**
3. You'll see options like:
 - Square
 - Circle
 - Triangle
 - Capsule

☞ **Choose: Square** (best for a basic player)

✓ A new object appears → rename it to **Player**

✓ Step 2: Understand What Got Added

When you create a Square sprite, Unity automatically adds:

- **Sprite Renderer** ✓
- **Transform** ✓

Now you need to add physics ☞

✓ Step 3: Add Components to Player

Select **Player** in Hierarchy → go to **Inspector (right side)**

Click **Add Component** and add:

1. Rigidbody2D

- Click **Add Component**
- Search: **Rigidbody2D**

Now set:

- **Gravity Scale = 3** (in Inspector)
-

2. Box Collider 2D

- Click **Add Component**
- Search: `Box Collider 2D`

✓ This gives collision to your player

PART 3: Adjust Camera (Where to set position)

Steps:

1. Click **Main Camera** in Hierarchy
2. Look at **Inspector panel (right side)**

You'll see **Transform component** like:

- Position → X | Y | Z

☞ Set:

- **X = 0**
- **Y = 0**
- **Z = -10**

✓ Just click on numbers and type values

PART 4: Create Ground (Detailed)

✓ Step 1: Add Ground

1. Right-click **Hierarchy**
2. → 2D Object → Sprites → Square

☞ Again choose **Square**

Rename it → **Ground**

✓ Step 2: Resize Ground

Select **Ground** → Inspector → Transform

Set **Scale**:

- **X = 10** (wide ground)
 - **Y = 1**
 - **Z = 1** (leave default)
-

✓ Step 3: Add Collider

Click **Add Component** → **Box Collider 2D**

✓ This makes it solid (player can stand)

✓ Step 4: Set Position (IMPORTANT)

☞ This is where you were confused

Where to set it:

1. Select **Ground**
2. In **Inspector** → **Transform**
3. Look for **Position**

You'll see:

- X | Y | Z

☞ Set:

- **Y = -3**

(Leave X = 0, Z = 0)

🎯 What this does

- Ground moves **down**
- Player stays above
- So player can **fall onto ground**

PART: Create Action Map & Move Action (Detailed)

✓ Step 1: Open Input Actions File

1. In **Project (Assets)** window
2. Find your file: `PlayerInputActions`
3. **Double-click it**

🔗 This opens the **Input Actions Editor** window

✓ Step 2: Create Action Map

You'll see 3 main columns:

- Left → **Action Maps**
 - Middle → **Actions**
 - Right → **Bindings**
-

🔗 Do this:

1. In **left column (Action Maps)**
2. Click + (**plus icon**)
3. Rename it to:

`Player`

✓ Press Enter

✓ Step 3: Add Action (Move)

Now look at **middle column (Actions)**

1. Make sure **Player** is selected
2. Click + (**plus icon**)
3. Rename action to:

Move

✓ Step 4: Set Action Type & Control Type

Now select **Move** → go to **right side Inspector panel**

Set these values:

◆ Action Type

Choose:

Value

☞ Means it gives continuous input (like movement)

◆ Control Type

Choose:

Vector2

☞ Means 2D input (X and Y direction)

✓ Step 5: Add Movement Keys (VERY IMPORTANT)

Now we define WASD keys ☞

1. Select Move

2. In **Bindings section** → click +
3. Choose:

Add 2D Vector Composite **or** Add Left/Right/Up/Down Composite

Now you'll see:

- Up

- Down
 - Left
 - Right
-

✓ Step 6: Assign Keys

Click each one and set:

- **Up** → **W**
- **Down** → **S**
- **Left** → **A**
- **Right** → **D**

Now Bind Each Direction

Up → W Key

1. Click "**Up <No Binding>**"
2. In Action Properties on right → click **Path field** (shows <No Binding>)
3. Click the **Listen** button OR click the keyboard icon
4. Press **W** on your keyboard
5. Select **W [Keyboard]** from the list

Down → S Key

1. Click "**Down <No Binding>**"
2. Click Path field → Listen → Press **S**
3. Select **S [Keyboard]**

Left → A Key

1. Click "**Left <No Binding>**"
2. Click Path field → Listen → Press **A**
3. Select **A [Keyboard]**

Right → D Key

1. Click "**Right <No Binding>**"
 2. Click Path field → Listen → Press **D**
 3. Select **D [Keyboard]**
-

 **Save When Done**

Click "**Save Asset**" in the top right corner

After saving it should look like:

- ▼ ● Move
 - ▼ 2D Vector
 - Up W [Keyboard]
 - Down S [Keyboard]
 - Left A [Keyboard]
 - Right D [Keyboard]

🎯 How to assign key:

1. Click binding (e.g., Up)
2. Click **Path**
3. Click **Listen**
4. Press key (W)

✓ Repeat for all

✓ Step 7: (Optional) Arrow Keys

You can add another binding:

1. Click + → **Add 2D Vector Composite again**
 2. Set:
 - Up → Up Arrow
 - Down → Down Arrow
 - Left → Left Arrow
 - Right → Right Arrow
-

✓ Step 8: Save

🖱️ Press **Ctrl + S** OR click **Save Asset**

PART 7: Generate C# Class (IMPORTANT)

✓ Step 1: Select Input Actions File

In **Project** → **Assets**

Click:

PlayerInputActions

✓ Step 2: Go to Inspector

On the right side (Inspector), you'll see options for Input Actions.

✓ Step 3: Enable “Generate C# Class”

✓ Tick:

Generate C# Class

✓ Step 4: Apply

Click:

Apply

🔑 What this does

Unity automatically creates:

PlayerInputActions.cs

This lets your script talk to the Input System.

☐ PART 8: Create Player Movement Script

✓ Step 1: Create Script

In Assets:

- Right-click
- → Create
- → MonoBehaviour Script

Name it:

PlayerController

✓ Step 2: Open Script

Double-click it

```
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.UI;
using TMPro;

public class PlayerController : MonoBehaviour
{
    [Header("Movement")]
    public float speed = 5f;
    public float jumpForce = 7f;

    [Header("UI References")]
    public TextMeshProUGUI scoreText;
    public TextMeshProUGUI gameOverText;
    public TextMeshProUGUI highScoreText;
    public Button restartButton;

    [Header("Game Settings")]
    public float fallDeathY = -5f;

    private Rigidbody2D rb;
    private Vector2 movement;
    private PlayerInputActions controls;

    private float score = 0f;
    private int highScore = 0;

    private bool isGameOver = false;
    private bool isGrounded = false;

    private float startX;

    void Awake()
    {
        controls = new PlayerInputActions();

        controls.Player.Move.performed += ctx =>
            movement = ctx.ReadValue<Vector2>();

        controls.Player.Move.canceled += ctx =>
```

```

        movement = Vector2.zero;
    }

    void OnEnable()
    {
        controls.Enable();
    }

    void OnDisable()
    {
        if (controls != null)
        {
            controls.Disable();
        }
    }

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();

        startX = transform.position.x;

        highScore = PlayerPrefs.GetInt("HighScore", 0);

        // Hide Game Over UI
        if (gameOverText)
            gameOverText.gameObject.SetActive(false);

        if (restartButton)
            restartButton.gameObject.SetActive(false);

        UpdateScoreUI();
    }

    void FixedUpdate()
    {
        if (isGameOver) return;

        // A and D movement
        rb.linearVelocity =
            new Vector2(movement.x * speed, rb.linearVelocity.y);
    }

    void Update()
    {
        if (isGameOver) return;

        // SPACE or W = Jump
        if ((Keyboard.current.spaceKey.wasPressedThisFrame ||
            Keyboard.current.wKey.wasPressedThisFrame)
            && isGrounded)
        {
            rb.linearVelocity =
                new Vector2(rb.linearVelocity.x, jumpForce);

            isGrounded = false;

            // Increase score after jump
            score += 10;

            UpdateScoreUI();
        }
    }

```

```

// S key = Fast fall
if (Keyboard.current.sKey.isPressed)
{
    rb.linearVelocity =
        new Vector2(rb.linearVelocity.x, -speed * 2);
}

// Score increases based on horizontal movement
float distanceTraveled =
    transform.position.x - startX;

if (distanceTraveled > score)
{
    score = distanceTraveled;

    UpdateScoreUI();
}

// Game Over after falling from plank
if (transform.position.y < fallDeathY)
{
    TriggerGameOver();
}
}

void UpdateScoreUI()
{
    if (scoreText)
        scoreText.text =
            "Score: " + Mathf.FloorToInt(score);

    if (highScoreText)
        highScoreText.text =
            "Best: " + highScore;
}

void TriggerGameOver()
{
    isGameOver = true;

    rb.linearVelocity = Vector2.zero;

    movement = Vector2.zero;

    // Save High Score
    if (Mathf.FloorToInt(score) > highScore)
    {
        highScore = Mathf.FloorToInt(score);

        PlayerPrefs.SetInt("HighScore", highScore);

        PlayerPrefs.Save();
    }

    // Show Game Over Text
    if (gameOverText)
    {
        gameOverText.gameObject.SetActive(true);

        gameOverText.text =
            "GAME OVER!\nScore: " +
            Mathf.FloorToInt(score) +
            "\nBest: " + highScore;
    }
}

```

```

    }

    // Show Restart Button
    if (restartButton)
        restartButton.gameObject.SetActive(true);
}

// Ground Detection
void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Ground"))
    {
        isGrounded = true;
    }
}

// Restart Button Function
public void RestartGame()
{
    UnityEngine.SceneManagement.SceneManager.LoadScene(
        UnityEngine.SceneManagement.SceneManager.GetActiveScene().name
    );
}
}

```

STEP 2 — Set Up the UI in Unity

Create Canvas

1. Hierarchy → Right-click → **UI → Canvas**
2. Canvas is created automatically with an **EventSystem**

Add Score Text

1. Right-click **Canvas** → **UI → Text - TextMeshPro**
2. Rename to `ScoreText`
3. In Inspector:
 - **Text** → `Score: 0`
 - **Anchor** → `Top Left`
 - **Pos X: 20, Pos Y: -20**
 - **Font Size: 28**

IMPORTANT SCENE SETUP

Ground

Make sure:

- Object name = `Ground`
- Has:

- Box Collider 2D
-

Player

Must have:

- Rigidbody2D
 - Box Collider2D
 - This script attached
-

HOW TO PLAY

1. Click ► Play
2. Use:
 - A/D → move
 - Space → jump
3. Try not to fall

STEP 3: Connect Score Text to Script

VERY IMPORTANT

1. Select Player object

2. In Inspector

You'll see:

Score Text = None

3. Drag ScoreText object

From Hierarchy → drag it into:

Score Text

field in PlayerController.

VERY IMPORTANT UNITY SETTINGS

✓ 1. Ground Setup

Select Ground object:

Set:

- Position Y = -3
 - Scale X = 10
 - Scale Y = 1
-

✓ 2. Add Ground Tag

Now assign that tag to Ground object. how?

Here's exactly how to assign the Ground tag in **Unity** 

✓ Step 1: Select Ground Object

In **Hierarchy**, click:

Ground

✓ Step 2: Find Tag at Top of Inspector

Look at the **top of Inspector panel**.

You'll see something like:

Tag: Untagged
Layer: Default

✓ Step 3: Create New Tag

Click:

Untagged ▼

Dropdown opens.

Select:

Add Tag...

✓ Step 4: Add Ground Tag

Now Inspector changes to Tags settings.

Under Tags:

Click:

+

Type:

Ground

Press Enter.

✓ Step 5: Assign Tag to Ground Object

Now:

1. Go back to Hierarchy
2. Select Ground again
3. Click:

Tag ▼

Now you'll see:

Ground

Click it ✓

.

✓ 3. Player Position

Select Player →

Set Position:

- $X = 0$
- $Y = 0$

So player starts above plank.

✓ 4. Rigidbody2D Settings

Player → Rigidbody2D

Set:

- Gravity Scale = 1.5
- Freeze Rotation Z = ✓